

ASSEMBLY



By: Maryam Eid :)

L02: Instruction Addressing & Execution

Absolute address

vs

Segment : offset address

عشان ال x86 processor الميموري بتاعته 1 MB فلو حسبناها هتبقى 2^{20} words يعني عشان أوصل لمكان في الميموري بوصله من خلال 20 bit address

ال absolute address ده بكون عارف فيه القيمة المباشرة للعنوان على طول وبروح له من غير ما أعمل على العنوان أي عمليات زي جمع أو غيره

هنا بقسم العنوان لجزئين:

- ال segment وده بيحدد بداية منطقة معينة في الميموري
- ال offset وده بيحدد المسافة أو الإزاحة جوا المنطقة دي

بحسب ازاي ال physical address أو العنوان الفعلي اللي أقدر أروح له على طول؟

$$(16 \times \text{segment}) + \text{offset} = \text{physical address}$$

يعني باخد قيمة ال segment وأضربها في 16 عشان هيكسا (زي ما بضرب في 10 بالديسمال) أو بمعنى آخر بزود صفر على اليمين وبعدين بضيف قيمة ال offset

Ex:

Segment = 2000H

Offset = 0040H

أول حاجة بنضرب ال Segment في 16 :

$$16 \times 2000H = 20000H$$

بعد كده هنضيف ال offset:

$$20000H + 0040H = 20040H$$

Program

البرنامج بيتكون من instructions, data, and functions (in assembly called subroutines) ولما بعمل له رن بيلود في الميموري، وعشان الوضع ما يبقاش خلطبيطة بالصلصة وكله يدخل في بعضه ويحصل override أو أي مشاكل تانية فالميموري عندي بتتنقسم لثلاثة أجزاء وده له فوائد من ضمنها إني هقدر أوصل لل instructions أو الداتا بسرعة كوني عارف هلاقي كل حاجة فين:

- **code segment** ده يحط فيه ال instructions زي MOV, ADD, and CALL
- **data segment** وده الجزء المخصص للبيانات اللي هيتعامل معاها البرنامج زي البيانات اللي في المتغيرات
- **stack segment** هو الجزء اللي بخرن فيه بيانات ال stack، وال stack ده عبارة عن data structure فهو مجرد ميموري برضو بس عمليات ال read وال write فيه بتبقى مختلفة شوية وبيستعمل مع ال functions عشان أخرن فيه حاجات مؤقتة وقت تنفيذ الكود زي ال return address بمعنى آخر أنا كنت واقف فين في البرنامج قبل ما أستدعي ال function أو ال arguments أو قيم ال registers أثناء تنفيذ ال function

Segments

- ال segment يبدأ عند paragraph boundary يعني عنوان بيقبل القسمة على 16 أو 10H وده معناه إن آخر رقم في العنوان هيبقى دائمًا 0 فمافيش داعي إنني أأخذ الصفر ده، فبالتالي 038E0H ممكن أكتبها 038EH أو 038E[0]H
- كل العناوين جوا ال segment بتبقى نسبية لبدايته، يعني لو ال segment بدأ عند 038EH هلاقي إن العناوين اللي حواه هتبقى زي 038E1H و 038E2H وهكذا
- Segment size = 64KB فبالتالي ال segment registers لازم يكون حجمها 16 bit عشان تقدر تأكسيس أي عنوان داخل ال segment

Registers

ال register هو وحدة تخزين صغيرة حجمه 16 bit ، بيبقى داخل ال CPU فبيقدر يوصل له أسرع من لو فضل رايح جاي لل RAM

Segment Registers

دي ال registers المسؤولة عن تحديد ال segments المختلفة في الميموري

- CS (Code Segment) بيحدد مكان الكود اللي بيتنفذه حاليًا
- DS (Data Segment) بيحدد مكان البيانات عامة
- SS (Stack Segment) بيحدد مكان ال Stack
- EX (Extra Segment) بيستخدم عادةً لعمليات إضافية زي ال string operations

Pointers registers

دي ال registers اللي بتساعد على تحديد (point) عناوين معينة في ال memory أو ال stack

- IP (Instruction Pointer) بيخزن offset عنوان ال instruction اللي عليها الدور في التنفيذ وال address الفعلي هو offset address : segment فبالتالي هيبقى CS : IP بمعنى تاني $CS \times 16 + IP$
- SP (Stack Pointer) بيخزن offset ال top of the stack اللي هو آخر مكان اتحفظت فيه داتا، بمعنى تاني بيقول لل CPU إيه المكان اللي هتبدأ منه read أو write على طول وبيبقى SS : SP
- BP (Base Pointer) ما بيتغيرش أثناء تنفيذ ال function عشان يفضل ثابت ويستخدمه البرنامج ك Reference للوصول للبيانات وبيبقى SS : BP

General purpose registers

- AX (Accumulator Register) بيستعمل بشكل أساسي في العمليات الحسابية زي الضرب والجمع وهكذا أو input / output operations وبيتقسم لجزئين (كذلك كل ال general purpose registers) كل جزء 8 bit
 - AH (the high byte)
 - AL (the low byte)

- **DX (Data Register)** يستعمل كذلك في ال input / output operations والعمليات الحسابية
 - لو ناتج الضرب كان أكبر من 6 bit وقتها النتيجة مش هتتخزن بس في ال AX، هتتوزع على AX و DX
 - في القسمة هتتخزن الجزء الصحيح في ال AX وباقي القسمة في ال DX
- **CX (Counter Register)** يستعمل في ال looping عن طريق تخزين عدد التكرارات فيه وبعد كل لفة القيمة هتقل بواحد لحد ما توصل للصفر ووقتها ال loop هتقف وممكن أذن فيه كمان القيمة اللي هيتم استعمالها في عملية ال shift
- **BX (Base Register)** هو ال general purpose register الوحيد اللي ممكن أستعمله في ال indirect addressing بمعنى إنني أقدر أستعمله وأذن فيه address معين وبعد كده ال CPU هيروح على ال address ده، ويستعمل برضو في العمليات الحسابية وال string addressing وال indexing مع ال SI وال DI

Index registers

بتساعد على الوصول للبيانات بشكل أسرع عن طريق ال index في ال strings أو ال arrays

- **SI (Source Index)** بيشار على المصدر يعني المكان اللي الداتا جاية منه
- **DI (Destination Index)** بيشار على الوجهة يعني المكان اللي الداتا هتروح له

Flag register

ال register ده بيحتوي على مجموعة من ال flags اللي بتدل على حالة ال CPU بعد عمليات معينة

- **OF (Overflow Flag)** بيبقى بواحد لو الناتج أكبر من المساحة اللي بحاول أأخزنه فيها
- **DF (Direction Flag)** معناه إنني بقرأ ال string من اليمين لليساار أو العكس
- **IF (Interrupt Flag)** يعني لو حصل interrupt هل ال CPU هيسمح به ويهتدله على طول ولا يتجاهله
- **TF (Trap Flag)** بيبقى بواحد لو عايز أنفذ الكود خطوة بخطوة (single step) وده بيساعد في ال debugging
- **SF (Sign Flag)** بيبقى بصفر لو الناتج موجب وبواحد لو سالب
- **ZF (Zero Flag)** معناه إن الناتج قيمته طلعت بصفر ولا لا، لو الناتج بصفر قيمة ال flag هتبقى بواحد
- **AF (Auxiliary Flag)** معناه هل حصل carry من الجزء ال low للجزء ال high في العمليات الحسابية
- **PF (Parity Flag)** بيدل على عدد الواحيد هل هو زوجي ولا فردي، لو كان زوجي هيبقى بواحد ولو فردي بصفر
- **CF (Carry Flag)** معناه هل حصل carry أو borrow ولا لا

Operating System

هو الوسيط ما بين البرنامج وال hardware وهو بيحاول يبسط الدنيا على ال user بدون ما يشغله بال low level details ويقدم له شوية services زي:

- **File management** اللي هي إدارة الملفات زي تخزين واسترجاع وحذف وكده
- **Input / output** يعني بيكون مسؤول عن التواصل ما بين جهاز الكمبيوتر والأجهزة الخارجية زي ال keyboard أو الطابعة وغيره

- Program loading يبلود البرنامج على الـ RAM عشان الـ CPU يقدر يرنه
- Memory management بيدير وينظم الـ RAM عشان يضمن إن كل برنامج بياخد بس الجزء اللي محتاجه بدون ما يتداخل مع برنامج تاني
- Interrupt handling المفروض لما يحصل interrupt الـ CPU بيوقف اللي بيعمله ويروح يهندله بس قبلها أصلًا
- الـ OS بيشوف أي interrupt له أولوية التنفيذ ويعمل context switch عشان يحفظ حالة الـ process اللي كانت شغالة عشان يرجع لها لما يخلص

BIOS Boot Process

هي مجموعة من الخطوات بيعملها الكمبيوتر أول ما بتضغط على زر الـ power عشان يشتغل

Turning on the power

Processor reset state

أول حاجة الـ RAM كله هيتصفر بمعنى أي داتا عليه هتتمسح لكن اللي في الـ ROM لا

Entry point

الـ CPU مش هيقدر ينفذ حاجة غير لما يلاقي أول instruction يبدأ من عندها ودي بتبقى أول instruction في الـ ROM عند الـ address FFFF0H وبعد كده بيد check على كل الـ ports عشان يتأكد إن كل الـ I / O devices تمام ومافيش مشكلة

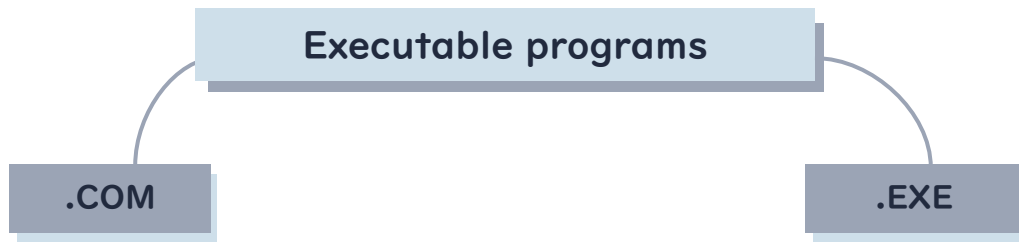
ITV & BIOS data area

بعد كده بيعمل initialization لحاجتين:

- الـ BIOS data area موجود عند الـ address 40H ودي فيها الـ status بتاعت كل I / O device متوصل بالكمبيوتر حاليًا
- الـ Interrupt Vector Table عبارة عن شوية addresses لـ instructions الـ CPU بيروح ينفذها عشان يهاندل interrupt وبتبقى متخزنة كـ offset : segment وفيه 256 4 byte addresses

System files

وأخيرًا عشان الـ OS يقدر ياخذ الـ control فالـ BIOS بيروح يدور في الـ disk في الـ system files على الـ boot strap loader اللي هو برنامج بيلود الـ system files في الميموري وبكده الـ OS بقى له الأكسيس



عبارة عن برنامج بسيط لما يبتجز له مكان في الميموري بيبقى عبارة عن segment واحد فيه ال code وال data وال stack

برنامج أكثر تعقيداً عشان بيبقى متقسم ل 3 segments اللي هم ال data وال code وال stack بس بيخلي البرنامج flexible أكثر

Program Loader

عشان ال CPU يقدر يرن البرنامج فلازم الأول يتعمل له load من ال disk للميموري واللي بيعمل كده اسمه ال program loader:

- أول حاجة بيجز 256 byte لل program segment prefix وده بيتخزن فيه حالة البرنامج ولو هعمل passing لداتا من خلال ال command line بتتخزن فيه كذلك
- وبعد كده بيلود أول address لل PSP في ال DS وال ES
- وأخيراً بيعمل initialization لل code, data, and stack segments

The Stack

الطبيعي إن لما برنامج بيتنفذ فهو بيدأ بتنفيذ ال main function وبعد كده لو احتاج ينفذ function تانية فهو بيروح لها ولما يخلصها بيرجع تاني لل main فالتالي هو بيحتاج يحفظ هو وقف فين بالضبط عشان يكمل من عنده وهنا بييجي دور ال stack:

- لما بحتاج أنفذ function تانية غير ال main فال CPU بيخزن ال return address في ال stack
- وكذلك ال function arguments بيتم تخزينها في ال stack
- حالة كل ال registers هتتخزن برضو في ال stack عشان لما يرجع لل main يقدر يرجع للحالة اللي كان فيها كأن شيئاً لم يكن

ال **stack frame** عبارة عن جزء في ال stack بيتم حجزه ل function معينة

Push & Pop

- عملية ال read في ال stack عن طريق ال pop وال write عن طريق ال push
- في ال push & pop لازم أتعامل مع داتا حجمها 16 bit
- ال stack pointer بيشاور على عنوان آخر داتا متخزنة في ال stack

مهم تفهم الطريقة بدون بصمحة عشان هي بتختلف لو كان ال stack طريقة تخزينه upword or downword

Push

بما إن ال SP بيشاور على آخر مكان فيه داتا فأنا محتاج الأول أخليه يشاور على مكان فاضي عشان أحط فيه الداتا اللي عندي فبيعلم له decrement بـ 2 bytes وبعد كده بخزن الداتا

Pop

هنا بيحصل العكس لأن بالفعل ال SP مشاور على الداتا اللي أنا عايزها فهورأها على طول وبعد كده أعمل increment لل SP بـ 2 bytes